

第二章 - 运动学

2.1 运动学初步 2.2 BODY与NED之间的变换 2.3 ECEF与NED之间的变换 2.4 ECEF与平地坐标之间的变换 2.5 BODY与FLOW之间的变换

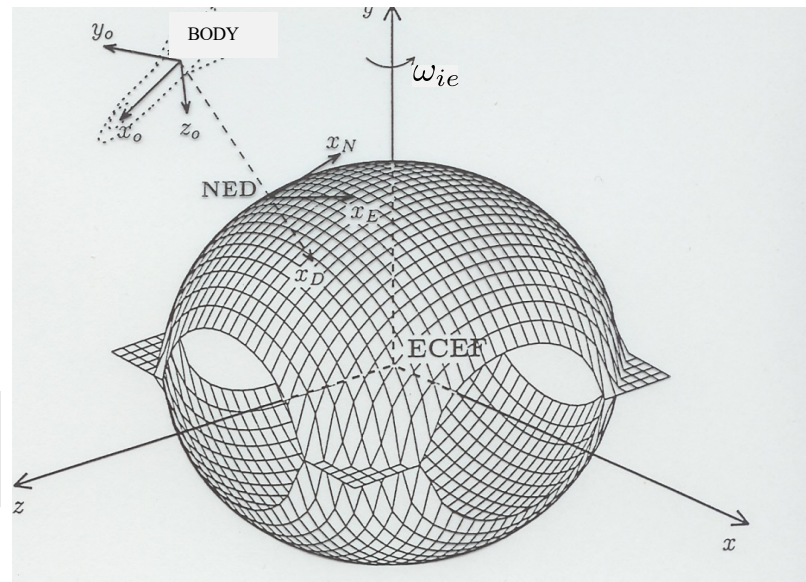
“动力学研究可以分为两部分：运动学，它只处理运动的几何方面；动力学，它是对引起运动的力的分析”

第2到第10章的总体目标

根据以下内容，以紧凑的矩阵-向量形式表示六自由度运动方程：

$$\dot{\eta} = J_{\Theta}(\eta)\nu$$

$$M\dot{\nu} + C(\nu)\nu + D(\nu)\nu + g(\eta) + g_0 = \tau + \tau_{\text{wind}} + \tau_{\text{wave}}$$



章节目标

- 理解地理坐标系NED、地心坐标系ECEF以及机体固定坐标系BODY。
- 了解什么是FLOW轴以及为什么我们在海上船舶和飞机上使用这些轴
- 能够写出将机体速度与NED位置相关联的微分方程，包括欧拉角和单位四元数的情况。
- 能够：
 - 将 ECEF (x^e, y^e, z^e) 位置转换为（经度，纬度，高度），反之亦然
 - 将（经度，纬度，高度）转换为平面地球坐标 (x^n, y^n, z^n)，反之亦然
- 定义、可视化并解释以下的用途：
 - 攻角 α
 - 侧滑角 β
 - 垂直螃蟹角 α_c
 - 水平螃蟹角 β_c
 - 航向角 ψ
 - 航向角 χ

2.1 参考系

地心参考系

{i}: 地心惯性参考系 (ECI)

$\{i\} = (x_i, y_i, z_i)$ 是地面导航的惯性参考系，即一个非加速参考系，在其中牛顿运动定律适用。

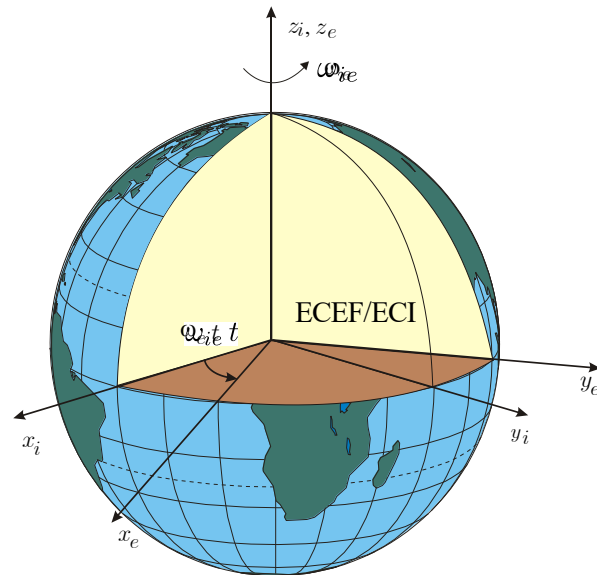
{e}: 地心地固参考系 (ECEF)

$\{e\} = (x_e, y_e, z_e)$ 的原点 o_e 固定在地球中心
但是这些轴相对于惯性坐标系ECI旋转，而ECI在空间中是固定的。

x_e - 位于赤道平面上，指向零度/本初子午线的轴；与格林威治天文台同经度
 y_e - 位于赤道平面上，完成右手坐标系的轴
 z_e - 指向地球自转轴的轴

地球的自转角速度是 $\omega_{ie} = 7.2921 \times 10^{-5}$ 弧度/秒，地球的旋转向量用 $\{e\}$ 表示为

$$\omega_{ie}^e = [0, 0, \omega_{ie}]^T$$



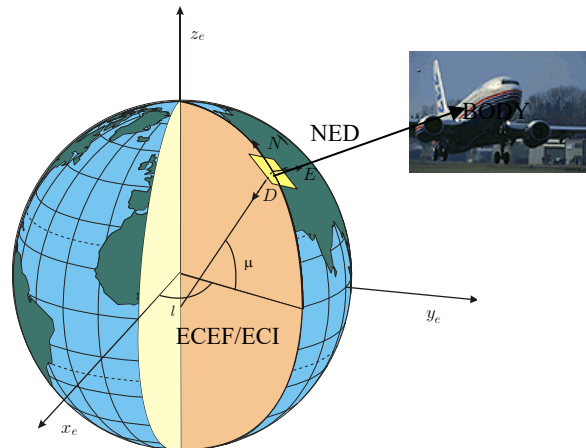
2.1 参考系（续）

地理参考框架（切平面）

地理参考框架通常选择为地球表面上的切平面。

- 陆地导航：地球表面的切平面随飞行器移动，其位置由随时间变化的经纬度值 (l, μ) 表示。切平面坐标系通常被旋转，使其轴指向北-东-下（NED）方向。
- 局部导航：切平面固定在常数值 (l_0, μ_0) ，位置相对于局部坐标原点计算。切平面的轴通常选择与NED轴重合。

“平地球导航”：位置精确到较小的地理区域（10公里 × 10公里）。



$\{n\}$: 东北下参考系（NED）

$\{n\} = (x_n, y_n, z_n)$ 的原点 o_n 定义为相对于地球参考椭球体（WGS 84），通常作为椭球体的切平面

x_n - 轴指向真北 y_n - 轴指向东方 z_n - 轴指向垂直于地表向下

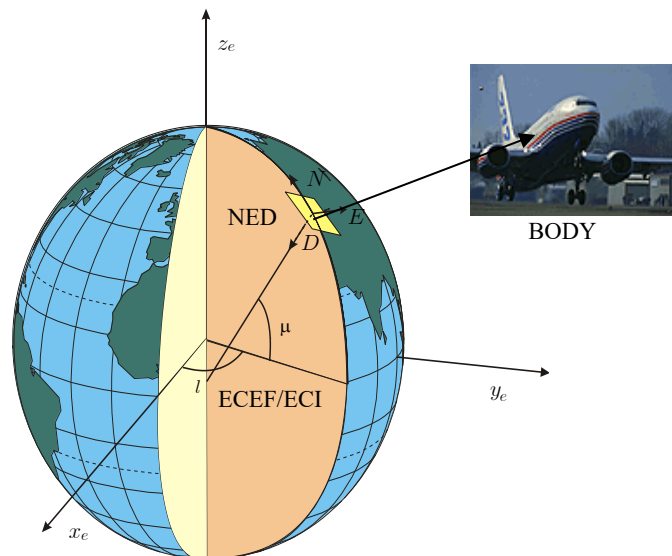
2.1 参考系 (续)

身体固定参考系

以机体为固定参考系 $\{b\} = (x_b, y_b, z_b)$ ，其原点为 o_b ，是一个固定在飞行器上的移动坐标系。

$\{b\}$: 机体固定参考系 (BODY)

对于海洋船舶，机体坐标轴的选择如下：
 x_b - 纵轴（从船尾指向船首）
 y_b - 横轴（指向右舷）
 z_b - 法向轴（从顶部指向底部）



$\{f\}$: 机体固定的流动轴参考系 (FLOW)

流向轴用于将 x 轴与飞行器的速度矢量对齐，使升力垂直于相对气流而阻力与之平行。从流向轴到机体轴的变换由两个主旋转定义，旋转角分别是迎角 α 和侧滑角 β 。流向轴的主要目的是简化升力和阻力的计算。

$$F_{\text{drag}} = \frac{1}{2} \rho V_r^2 S C_D(\alpha)$$

$$F_{\text{lift}} = \frac{1}{2} \rho V_r^2 S C_L(\alpha)$$

$$\begin{bmatrix} X \\ Z \end{bmatrix} = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix} \begin{bmatrix} -F_{\text{drag}} \\ -F_{\text{lift}} \end{bmatrix}$$

2.1 车体固定参考点

最重要的参考点

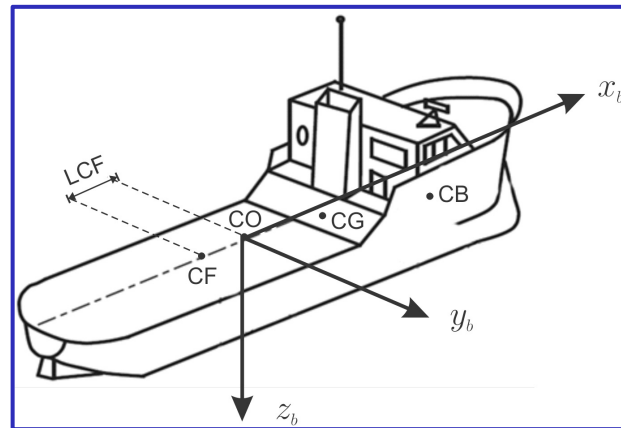
CO - Coordinate origin o_b of the body-fixed frame $\{b\}$. CO also represents the coordinate origin of the guidance, navigation and motion control systems.

以下随时间变化的点是相对于 CO 表示的

CG - Center of gravity located at $\mathbf{r}_{bg}^b = [x_g, y_g, z_g]^\top$ relative to the CO.

CB - Center of buoyancy located at $\mathbf{r}_{bb}^b = [x_b, y_b, z_b]^\top$ relative to the CO.

CF - Center of flotation located at $\mathbf{r}_{bf}^b = [x_f, y_f, z_f]^\top$ relative to the CO.



CF位于距离CO在 x 方向LCF的地方

浮心是平静水面上水面面积 A_{wp} 的质心。对于小角度（线性理论），船舶将围绕该点横摇和纵摇。

2.1 广义坐标

对于不受任何运动约束的海上船只

Number of independent (generalized) coordinates = DOFs

广义坐标一词指的是描述飞行器相对于某些参考构型的配置的参数。

对于海洋船舶，广义位置和速度向量为

$$\boldsymbol{\eta} = [x^n, y^n, z^n, \phi, \theta, \psi]^\top$$

$$\dot{\boldsymbol{\eta}} = [\dot{x}^n, \dot{y}^n, \dot{z}^n, \dot{\phi}, \dot{\theta}, \dot{\psi}]^\top$$

这些量都是用NED表示的。把速度表示出来是有利的
在BODY框架中的工艺。换句话说

$$\boldsymbol{\nu} = [u, v, w, p, q, r]^\top$$

接下来的页面将推导NED和BODY中速度向量之间的关系。

2.1 向量表示法 (Fossen 2021)

p_{eb}^e	position of the CO relative to o_e expressed in $\{e\}$
p_{nb}^n	position of the CO relative to o_n expressed in $\{n\}$
v_{nb}^e	linear velocity of the CO relative to o_n expressed in $\{e\}$
v_{nb}^n	linear velocity of the CO relative to o_n expressed in $\{n\}$
ω_{nb}^b	angular velocity of $\{b\}$ with respect to $\{n\}$ expressed in $\{b\}$
f_b^n	force with line of action through the point CO expressed in $\{n\}$
m_b^n	moment about the point CO expressed in $\{n\}$
Θ_{nb}	Euler angles from $\{b\}$ to $\{n\}$
q_b^n	Unit quaternion from $\{b\}$ to $\{n\}$

ECEF position $p_{eb}^e = \begin{bmatrix} x^e \\ y^e \\ z^e \end{bmatrix} \in \mathbb{R}^3$

NED position $p_{nb}^n = \begin{bmatrix} x^n \\ y^n \\ z^n \end{bmatrix} \in \mathbb{R}^3$

Body-fixed linear velocity $v_{nb}^b = \begin{bmatrix} u \\ v \\ w \end{bmatrix} \in \mathbb{R}^3$

Body-fixed force $f_b^b = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \in \mathbb{R}^3$

Longitude and latitude $\Theta_{en} = \begin{bmatrix} l \\ \mu \end{bmatrix} \in \mathbb{T}^2$

Attitude (Euler angles) $\Theta_{nb} = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix} \in \mathbb{T}^3$

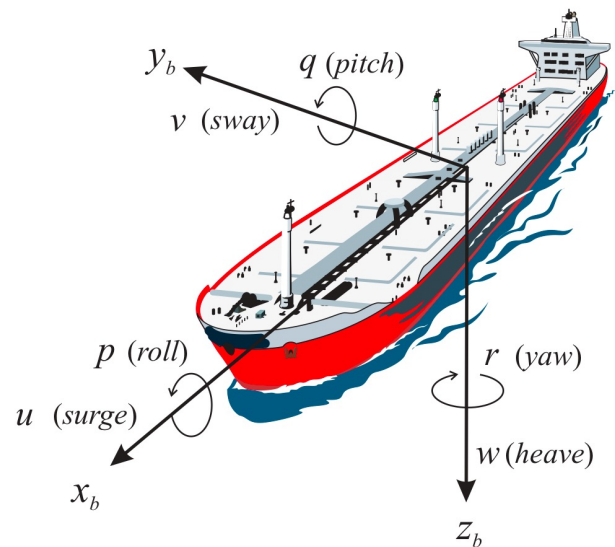
Body-fixed angular velocity $\omega_{nb}^b = \begin{bmatrix} p \\ q \\ r \end{bmatrix} \in \mathbb{R}^3$

Body-fixed moment $m_b^b = \begin{bmatrix} K \\ M \\ N \end{bmatrix} \in \mathbb{R}^3$

2.1 六自由度（6-DOF）运动

6自由度（6-DOF）指的是刚体在三维空间中的运动自由度。它描述了物体相对于自身固定坐标轴的平移和旋转方式。它不用于NED坐标系。

BODY				NED
DOF		Forces and moments	Linear and angular velocities	Positions and Euler angles
1	Motions in the x_b -direction (surge)	X	u	x^n
2	Motions in the y_b -direction (sway)	Y	v	y^n
3	Motions in the z_b -direction (heave)	Z	w	z^n
4	Rotation about the x_b -axis (roll)	K	p	ϕ
5	Rotation about the y_b -axis (pitch)	M	q	θ
6	Rotation about the z_b -axis (yaw)	N	r	ψ



该符号采用自：

SNAME (1950)。处理潜入流体中的物体运动的命名法。 *The Society of Naval Architects and Marine Engineers, Technical and Research Bulletin No. 1-5, April 1950, pp. 1-15.*

2.1 摘要：6自由度向量

ECEF position	$\mathbf{p}_{eb}^e = \begin{bmatrix} x^e \\ y^e \\ z^e \end{bmatrix} \in \mathbb{R}^3$	Longitude and latitude	$\Theta_{en} = \begin{bmatrix} l \\ \mu \end{bmatrix} \in \mathbb{T}^2$
NED position	$\mathbf{p}_{nb}^n = \begin{bmatrix} x^n \\ y^n \\ z^n \end{bmatrix} \in \mathbb{R}^3$	Attitude (Euler angles)	$\Theta_{nb} = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix} \in \mathbb{T}^3$
Body-fixed linear velocity	$\mathbf{v}_{nb}^b = \begin{bmatrix} u \\ v \\ w \end{bmatrix} \in \mathbb{R}^3$	Body-fixed angular velocity	$\boldsymbol{\omega}_{nb}^b = \begin{bmatrix} p \\ q \\ r \end{bmatrix} \in \mathbb{R}^3$
Body-fixed force	$\mathbf{f}_b^b = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \in \mathbb{R}^3$	Body-fixed moment	$\mathbf{m}_b^b = \begin{bmatrix} K \\ M \\ N \end{bmatrix} \in \mathbb{R}^3$

坐标系 {b}、{n} 和 {e}

上标表示：

- b: 在 {b} 中表示的向量。
- n: 在 {n} 中表示的向量。
- e: 在 {e} 中表示的向量。

下标 nb 意思是：

- 角速度：参考框架 {n} 下框架 {b} 的旋转速度。
- 线速度：参考框架 {n} 原点下框架 {b} 原点的平移速度。
- 欧拉角：参考框架 {n} 下框架 {b} 的方向。
- 位置：从原点 {n} 到原点 {b} 的位置向量。

以 {b} 表示的6自由度广义位置、速度和力向量

$$\boldsymbol{\eta} = \begin{bmatrix} \mathbf{p}_{nb}^n \\ \Theta_{nb} \end{bmatrix}, \quad \boldsymbol{\nu} = \begin{bmatrix} \mathbf{v}_{nb}^b \\ \boldsymbol{\omega}_{nb}^b \end{bmatrix}, \quad \boldsymbol{\tau} = \begin{bmatrix} \mathbf{f}_b^b \\ \mathbf{m}_b^b \end{bmatrix}$$

广义向量上不使用上标或下标。

2.2 BODY与NED之间的转换

Orthogonal matrices of order 3:

$$O(3) = \{\mathbf{R} | \mathbf{R} \in \mathbb{R}^{3 \times 3}, \quad \mathbf{R}\mathbf{R}^\top = \mathbf{R}^\top\mathbf{R} = \mathbf{I}\}$$

Special orthogonal group of order 3:

$$SO(3) = \{\mathbf{R} | \mathbf{R} \in \mathbb{R}^{3 \times 3}, \quad \mathbf{R} \text{ is orthogonal and } \det \mathbf{R} = 1\}$$

Rotation matrix:

$$\mathbf{R}\mathbf{R}^\top = \mathbf{R}^\top\mathbf{R} = \mathbf{I}, \quad \det \mathbf{R} = 1$$

Since $\mathbf{R}$ is orthogonal, $\mathbf{R}^{-1} = \mathbf{R}^\top$

Example:

$$\mathbf{v}^{\text{to}} = \mathbf{R}_{\text{from}}^{\text{to}} \mathbf{v}^{\text{from}}$$

2.2 BODY 与 NED 之间的变换 (续)

叉积运算符作为矩阵-向量乘法:

$$\mathbf{S}(\boldsymbol{\lambda}) = -\mathbf{S}^\top(\boldsymbol{\lambda}) = \begin{bmatrix} 0 & -\lambda_3 & \lambda_2 \\ \lambda_3 & 0 & -\lambda_1 \\ -\lambda_2 & \lambda_1 & 0 \end{bmatrix}, \quad \boldsymbol{\lambda} = \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \lambda_3 \end{bmatrix}$$

其中 $\mathbf{S}$ 是一个斜对称矩阵

逆运算符表示为:

$$\boldsymbol{\lambda} = \text{vex}(\mathbf{S}(\boldsymbol{\lambda}))$$

Matlab:

The cross-product operator is included in the MSS toolbox as `Smtx.m`. Hence, the cross-product $\mathbf{b} = \mathbf{S}(\boldsymbol{\lambda})\mathbf{a}$ can be computed as

```
S = Smtx(lambda);
b = S * a;
```

The inverse operation is

```
lambda = vex(S);
```

2.2 BODY 与 NED 之间的变换 (续)

欧拉旋转定理

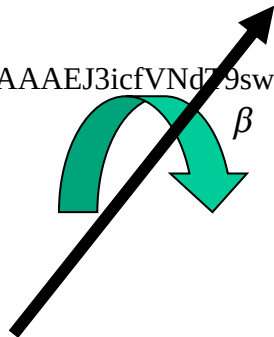
latexit sha1_base64"p0l11X8yX/zS4T4QlNcEvrR3Wok"AAACZ3icbVFbSyMxGM2Mumq9tF4QwZesRaisW2ZaUV8E0Rd90

$!b_n ! "n!b_{b/n} " "n_b \# ! "_{\lambda,\beta} \quad \$\$ \$\$ \$\$$

$!!!\lambda_1 \lambda_2 \lambda_3^\top \%!\% !!!$

哪里

乳胶sha1_base64 “5xO757M6pbmEiMWl1ERB/F/qs” AAAEJ3icfVNdz9swFDXJPlj3QRmPe7GoNjEVqjiZYC9IaHvhEaYV



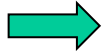
2.2 Euler Angle Transformation

Three principal rotations

$$\lambda = [0, 0, 1]^T \quad \beta = \psi$$

$$\lambda = [0, 1, 0]^T \quad \beta = \theta$$

$$\lambda = [1, 0, 0]^T \quad \beta = \phi$$

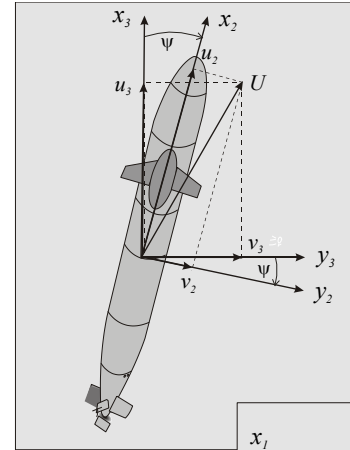


The ZXZ sequence is the proper Euler angle rotation sequence used in robotics, but for aircraft and spacecraft, we use the Tait–Bryan ZYX sequence (yaw-pitch-roll), so that the singularity occurs at ± 90 degrees in pitch.

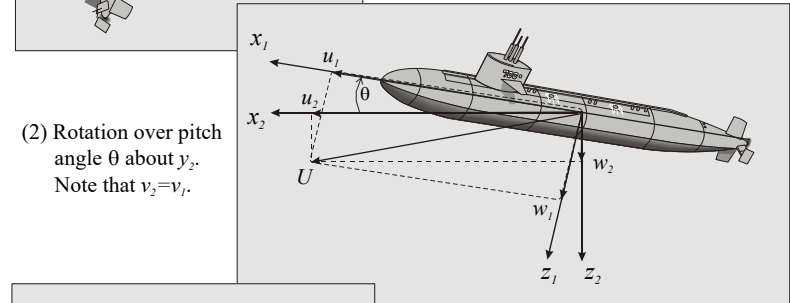
$$\mathbf{R}_{z,\psi} = \begin{bmatrix} c\psi & s\psi & 0 \\ s\psi & c\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{R}_{y,\theta} = \begin{bmatrix} c\theta & 0 & s\theta \\ 0 & 1 & 0 \\ s\theta & 0 & c\theta \end{bmatrix}$$

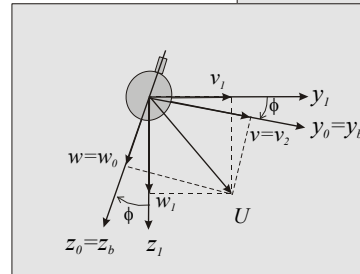
$$\mathbf{R}_{x,\phi} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c\phi & s\phi \\ 0 & s\phi & c\phi \end{bmatrix}$$



(1) Rotation over yaw angle ψ about z_3 . Note that $w_3 = w_2$.



(2) Rotation over pitch angle θ about y_2 . Note that $v_2 = v_1$.



(3) Rotation over roll angle ϕ about x_1 . Note that $u_1 = u_2$.

2.2 欧拉角变换 (续)

线速度变换 (zyx 约定)

$$\dot{\mathbf{p}}_{nb}^n = \mathbf{R}(\boldsymbol{\Theta}_{nb}) \mathbf{v}_{nb}^b$$

欧拉角旋转矩阵

$$\mathbf{R}_b^n := \mathbf{R}(\boldsymbol{\Theta}_{nb})$$

$$\mathbf{R}_b^n := \mathbf{R}_{z,\psi} \mathbf{R}_{y,\theta} \mathbf{R}_{x,\phi}$$

$$(\mathbf{R}_b^n)^{-1} = \mathbf{R}_n^b = \mathbf{R}_{x,\phi}^\top \mathbf{R}_{y,\theta}^\top \mathbf{R}_{z,\psi}^\top$$

$$\mathbf{R}(\boldsymbol{\Theta}_{nb}) = \begin{bmatrix} c\psi c\theta & -s\psi c\phi + c\psi s\theta s\phi & s\psi s\phi + c\psi c\phi s\theta \\ s\psi c\theta & c\psi c\phi + s\phi s\theta s\psi & -c\psi s\phi + s\theta s\psi c\phi \\ -s\theta & c\theta s\phi & c\theta c\phi \end{bmatrix}$$

Matlab:

The rotation matrix $\mathbf{R}(\boldsymbol{\Theta}_{nb})$ is implemented in the MSS toolbox as

```
R = Rzyx(phi,theta,psi)
```

小角 le approximation

$$\mathbf{R}(\delta\boldsymbol{\Theta}_{nb}) \approx \mathbf{I}_3 + \mathbf{S}(\delta\boldsymbol{\Theta}_{nb}) = \begin{bmatrix} 1 & -\delta\psi & \delta\theta \\ \delta\psi & 1 & -\delta\phi \\ -\delta\theta & \delta\phi & 1 \end{bmatrix}$$

2.2 欧拉角变换（续）

NED 位置（连续时间和离散时间）

$$\dot{\mathbf{p}}_{nb}^n = \mathbf{R}(\boldsymbol{\Theta}_{nb}) \mathbf{v}_{nb}^b$$



欧拉法，采样时间
h

$$\mathbf{p}_{nb}^n[k+1] = \mathbf{p}_{nb}^n[k] + h \mathbf{R}(\boldsymbol{\Theta}_{nb}[k]) \mathbf{v}_{nb}^b[k]$$

然而，由于数值精度和稳定性，建议使用更高阶的方法，例如 RK4

分量式

$$\begin{aligned} \dot{x}^n &= u \cos(\psi) \cos(\theta) + v [\cos(\psi) \sin(\theta) \sin(\phi) - \sin(\psi) \cos(\phi)] \\ &\quad + w [\sin(\psi) \sin(\phi) + \cos(\psi) \cos(\phi) \sin(\theta)] \\ \dot{y}^n &= u \sin(\psi) \cos(\theta) + v [\cos(\psi) \cos(\phi) + \sin(\phi) \sin(\theta) \sin(\psi)] \\ &\quad + w [\sin(\theta) \sin(\psi) \cos(\phi) - \cos(\psi) \sin(\phi)] \\ \dot{z}^n &= -u \sin(\theta) + v \cos(\theta) \sin(\phi) + w \cos(\theta) \cos(\phi) \end{aligned}$$

2.2 欧拉角变换 (续)

角速度变换 (zyx 约定)

$$\dot{\Theta}_{nb} = T(\Theta_{nb})\omega_{nb}^b \quad \omega_{nb}^b = \begin{bmatrix} \dot{\phi} \\ 0 \\ 0 \end{bmatrix} + R_{x,\phi}^\top \begin{bmatrix} 0 \\ \dot{\theta} \\ 0 \end{bmatrix} + R_{x,\phi}^\top R_{y,\theta}^\top \begin{bmatrix} 0 \\ 0 \\ \dot{\psi} \end{bmatrix} := T^{-1}(\Theta_{nb})\dot{\Theta}_{nb}$$

哪里

$$T^{-1}(\Theta_{nb}) = \begin{bmatrix} 1 & 0 & -s\theta \\ 0 & c\phi & c\theta s\phi \\ 0 & -s\phi & c\theta c\phi \end{bmatrix} \implies T(\Theta_{nb}) = \begin{bmatrix} 1 & s\phi t\theta & c\phi t\theta \\ 0 & c\phi & -s\phi \\ 0 & s\phi/c\theta & c\phi/c\theta \end{bmatrix}$$

小角度近似

$$T_{\Theta}(\delta\Theta_{nb}) \approx \begin{bmatrix} 1 & 0 & \delta\theta \\ 0 & 1 & -\delta\phi \\ 0 & \delta\phi & 1 \end{bmatrix}$$

奇异点在 $\theta \rightarrow \pm \pi/2$ 处 $\Theta_{nb}^\top$

$\Theta_{nb}^\top \neq$

2.2 欧拉角变换 (续)

欧拉角姿态表示

欧拉角的常微分方程

$$\dot{\Theta}_{nb} = T(\Theta_{nb}) \omega_{nb}^b$$

分量式

$$\dot{\phi} = p + q \sin(\phi) \tan(\theta) + r \cos(\phi) \tan(\theta)$$

$$\dot{\theta} = q \cos(\phi) - r \sin(\phi)$$

$$\dot{\psi} = q \frac{\sin(\phi)}{\cos(\theta)} + r \frac{\cos(\phi)}{\cos(\theta)}, \quad \theta \neq \pm 90^\circ$$

Matlab:

The transformation matrix $T(\Theta_{nb})$ is implemented in the MSS toolbox as

```
T = Tzyx(phi,theta)
```

旋转矩阵的常微分方程

$$\dot{R}_b^n = R_b^n S(\omega_{nb}^b)$$

哪里

$$S(\omega_{nb}^b) = \begin{bmatrix} 0 & -r & q \\ r & 0 & -p \\ -q & p & 0 \end{bmatrix}$$

必须与算法结合使用
从旋转矩阵计算欧拉角

R_b^n

$$\Theta_{nb} = [\phi, \theta, \psi]^\top$$

2.2 欧拉角变换 (续)

摘要：六自由度运动学方程

$$\dot{\eta} = J_{\Theta}(\eta)\nu$$

$$\begin{bmatrix} \dot{p}_{nb}^n \\ \dot{\Theta}_{nb} \end{bmatrix} = \begin{bmatrix} R(\Theta_{nb}) & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & T(\Theta_{nb}) \end{bmatrix} \begin{bmatrix} v_{nb}^b \\ \omega_{nb}^b \end{bmatrix}$$

在 θ 处具有奇异性的
三参数表示!!
 $\phi! \theta! \psi''^T$

分量式

$$\begin{aligned} \dot{x}^n &= u \cos(\psi) \cos(\theta) + v [\cos(\psi) \sin(\theta) \sin(\phi) - \sin(\psi) \cos(\phi)] \\ &\quad + w [\sin(\psi) \sin(\phi) + \cos(\psi) \cos(\phi) \sin(\theta)] \\ \dot{y}^n &= u \sin(\psi) \cos(\theta) + v [\cos(\psi) \cos(\phi) + \sin(\phi) \sin(\theta) \sin(\psi)] \\ &\quad + w [\sin(\theta) \sin(\psi) \cos(\phi) - \cos(\psi) \sin(\phi)] \\ \dot{z}^n &= -u \sin(\theta) + v \cos(\theta) \sin(\phi) + w \cos(\theta) \cos(\phi) \\ \dot{\phi} &= p + q \sin(\phi) \tan(\theta) + r \cos(\phi) \tan(\theta) \\ \dot{\theta} &= q \cos(\phi) - r \sin(\phi) \\ \dot{\psi} &= q \frac{\sin(\phi)}{\cos(\theta)} + r \frac{\cos(\phi)}{\cos(\theta)}, \quad \theta \neq \pm 90^\circ \end{aligned}$$

```
phi = deg2rad(10)
theta = deg2rad(20)
psi = deg2rad(30)

R = Rzyx(phi, theta, psi)

T = Tzyx(phi, theta)
```

```
phi =
    0.1745
theta =
    0.3491
psi =
    0.5236
R =
    0.8138    -0.4410    0.3785
    0.4698     0.8826    0.0180
   -0.3420     0.1632    0.9254
T =
    1.0000    0.0632    0.3584
         0     0.9848   -0.1736
         0     0.1848    1.0480
```

2.2 单元四元数

四参数表示

$$Q = \{\mathbf{q} | \mathbf{q}^T \mathbf{q} = 1, \mathbf{q} = [\eta, \boldsymbol{\varepsilon}^T]^T, \boldsymbol{\varepsilon} \in \mathbb{R}^3 \text{ and } \eta \in \mathbb{R}\}$$

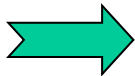
- 避免了被称为万向节锁的三参数欧拉角表示的表示奇异性- 涉及较少的三角计算，提高了稳定性- 四元数插值相比欧拉角能够产生更平滑的过渡- 四元数乘法比组合欧拉角更高效、更稳定

单位四元数（欧拉参数）旋转矩阵（Chou 1992）

$$\mathbf{R}_{\beta, \lambda} = \mathbf{I}_{3 \times 3} + \sin \beta \mathbf{S}(\boldsymbol{\lambda}) + (1 - \cos \beta) \mathbf{S}^2(\boldsymbol{\lambda})$$

$$\eta = \cos \frac{\beta}{2}$$

$$\boldsymbol{\varepsilon} = [\varepsilon_1, \varepsilon_2, \varepsilon_3]^T = \boldsymbol{\lambda} \sin \frac{\beta}{2}$$



$$\mathbf{q} = \begin{bmatrix} \eta \\ \varepsilon_1 \\ \varepsilon_2 \\ \varepsilon_3 \end{bmatrix} = \begin{bmatrix} \cos \frac{\beta}{2} \\ \boldsymbol{\lambda} \sin \frac{\beta}{2} \end{bmatrix} \in Q$$

$$\mathbf{R}(\mathbf{q}_b^n) := \mathbf{I}_3 + 2\eta \mathbf{S}(\boldsymbol{\varepsilon}) + 2\mathbf{S}^2(\boldsymbol{\varepsilon})$$

Chou, J. C. K. (1992)。四元数运动学与动力学微分方程。《IEEE机器人与自动化汇刊》8(1), 53–64。

2.2 单元四元数 (续)

线速度变换

$$\dot{\mathbf{p}}_{nb}^n = \mathbf{R}(\mathbf{q}_b^n) \mathbf{v}_{nb}^b \quad \mathbf{R}^{-1}(\mathbf{q}_b^n) = \mathbf{R}^\top(\mathbf{q}_b^n)$$

$$\mathbf{R}(\mathbf{q}_b^n) = \begin{bmatrix} 1 - 2(\varepsilon_2^2 + \varepsilon_3^2) & 2(\varepsilon_1\varepsilon_2 - \varepsilon_3\eta) & 2(\varepsilon_1\varepsilon_3 + \varepsilon_2\eta) \\ 2(\varepsilon_1\varepsilon_2 + \varepsilon_3\eta) & 1 - 2(\varepsilon_1^2 + \varepsilon_3^2) & 2(\varepsilon_2\varepsilon_3 - \varepsilon_1\eta) \\ 2(\varepsilon_1\varepsilon_3 - \varepsilon_2\eta) & 2(\varepsilon_2\varepsilon_3 + \varepsilon_1\eta) & 1 - 2(\varepsilon_1^2 + \varepsilon_2^2) \end{bmatrix}$$

分量形式 (北东天坐标位置)

$$\dot{x}^n = u(1 - 2\varepsilon_2^2 - 2\varepsilon_3^2) + 2v(\varepsilon_1\varepsilon_2 - \varepsilon_3\eta) + 2w(\varepsilon_1\varepsilon_3 + \varepsilon_2\eta)$$

$$\dot{y}^n = 2u(\varepsilon_1\varepsilon_2 + \varepsilon_3\eta) + v(1 - 2\varepsilon_1^2 - 2\varepsilon_3^2) + 2w(\varepsilon_2\varepsilon_3 - \varepsilon_1\eta)$$

$$\dot{z}^n = 2u(\varepsilon_1\varepsilon_3 - \varepsilon_2\eta) + 2v(\varepsilon_2\varepsilon_3 + \varepsilon_1\eta) + w(1 - 2\varepsilon_1^2 - 2\varepsilon_2^2)$$

Matlab:

The quaternion rotation matrix is easily computed by using the MSS toolbox commands

```
q = [eta, eps1, eps2, eps3];
R = Rquat(q);
```

必须在约束 η !! ε_1^2 !
 ε_2^2 ! ε_3^2 下进行积分

2.2 单元四元数 (续)

角速度变换

$$\dot{\mathbf{q}}_b^n = \frac{1}{2} \begin{bmatrix} -\boldsymbol{\varepsilon}^\top \\ \eta \mathbf{I}_3 + \mathbf{S}(\boldsymbol{\varepsilon}) \end{bmatrix} \boldsymbol{\omega}_{nb}^b$$

$$:= \mathbf{T}(\mathbf{q}_b^n) \boldsymbol{\omega}_{nb}^b$$

$$\mathbf{T}(\mathbf{q}_b^n) = \frac{1}{2} \begin{bmatrix} -\varepsilon_1 & -\varepsilon_2 & -\varepsilon_3 \\ \eta & -\varepsilon_3 & \varepsilon_2 \\ \varepsilon_3 & \eta & -\varepsilon_1 \\ -\varepsilon_2 & \varepsilon_1 & \eta \end{bmatrix}, \quad \mathbf{T}^\top(\mathbf{q}_b^n) \mathbf{T}(\mathbf{q}_b^n) = \frac{1}{4} \mathbf{I}_3$$

使用四元数乘积的替代表示

$$\dot{\mathbf{q}}_b^n = \frac{1}{2} \mathbf{q}_b^n \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega}_{nb}^b \end{bmatrix} \quad \mathbf{q}_1 \otimes \mathbf{q}_2 = \begin{bmatrix} \eta_1 \eta_2 - \boldsymbol{\varepsilon}_1^\top \boldsymbol{\varepsilon}_2 \\ \eta_1 \boldsymbol{\varepsilon}_2 + \eta_2 \boldsymbol{\varepsilon}_1 + \mathbf{S}(\boldsymbol{\varepsilon}_1) \boldsymbol{\varepsilon}_2 \end{bmatrix}$$

Matlab:

The transformation matrix $\mathbf{T}(\mathbf{q}_b^n)$ is implemented in the MSS toolbox as

`T = Tquat(q)`

对于多一个参数的价格来说是非奇异的，但单位四元数在数值上是高效的

```
quat = euler2q(phi, theta, psi)
```

```
Rq = Rquat(quat)
```

```
Tq = Tquat(quat)
```

```
quat =
```

```
0.9515
```

```
0.0381
```

```
0.1893
```

```
0.2393
```

```
Rq =
```

```
0.8138
```

```
-0.4410
```

```
0.3785
```

```
0.4698
```

```
0.8826
```

```
0.0180
```

```
-0.3420
```

```
0.1632
```

```
0.9254
```

```
Tq =
```

```
-0.0191
```

```
-0.0947
```

```
-0.1196
```

```
0.4758
```

```
-0.1196
```

```
0.0947
```

```
0.1196
```

```
0.4758
```

```
-0.0191
```

```
-0.0947
```

```
0.0191
```

```
0.4758
```

2.2 单元四元数（续）

摘要：6自由度运动学方程（7个常微分方程）

$$\dot{\eta} = J_q(\eta)\nu$$

$$\Updownarrow$$

$$\begin{bmatrix} \dot{p}_{nb}^n \\ \dot{q}_b^n \end{bmatrix} = \begin{bmatrix} R(q_b^n) & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{4 \times 3} & T(q_b^n) \end{bmatrix} \begin{bmatrix} v_{nb}^b \\ \omega_{nb}^b \end{bmatrix}$$

$$!! \eta! \varepsilon_1! \varepsilon_2! \varepsilon_3!^\top$$

$$\eta = [x^n, y^n, z^n, \eta, \varepsilon_1, \varepsilon_2, \varepsilon_3]^\top$$

分量式

$$\dot{x}^n = u(1 - 2\varepsilon_2^2 - 2\varepsilon_3^2) + 2v(\varepsilon_1\varepsilon_2 - \varepsilon_3\eta) + 2w(\varepsilon_1\varepsilon_3 + \varepsilon_2\eta)$$

$$\dot{y}^n = 2u(\varepsilon_1\varepsilon_2 + \varepsilon_3\eta) + v(1 - 2\varepsilon_1^2 - 2\varepsilon_3^2) + 2w(\varepsilon_2\varepsilon_3 - \varepsilon_1\eta)$$

$$\dot{z}^n = 2u(\varepsilon_1\varepsilon_3 - \varepsilon_2\eta) + 2v(\varepsilon_2\varepsilon_3 + \varepsilon_1\eta) + w(1 - 2\varepsilon_1^2 - 2\varepsilon_2^2)$$

$$\dot{\eta} = -\frac{1}{2}(\varepsilon_1 p + \varepsilon_2 q + \varepsilon_3 r)$$

$$\dot{\varepsilon}_1 = \frac{1}{2}(\eta p - \varepsilon_3 q + \varepsilon_2 r)$$

$$\dot{\varepsilon}_2 = \frac{1}{2}(\varepsilon_3 p + \eta q - \varepsilon_1 r)$$

$$\dot{\varepsilon}_3 = \frac{1}{2}(-\varepsilon_2 p + \varepsilon_1 q + \eta r)$$

4 参数表示 非奇异，但需要再一个常微分方程。

2.2 Unit Quaternions (cont.)

Discretization of the unit quaternion differential equation

% Exact discretization is much more accurate than Euler's method

% MSS supports both methods thanks to Tquat.m

w = [p q r]';

% Euler intrgration

q = q + h * Tquat(q) * w;

q = q / sqrt(q' * q);

% Exact discretization

q = expm(Tquat(w) * h) * q;

q = q / sqrt(q' * q);

$$\dot{q}_b^n = \frac{1}{2} \begin{bmatrix} -\varepsilon^\top \\ \eta I_3 + S(\varepsilon) \end{bmatrix} \omega_{nb}^b$$

$$:= T(q_b^n) \omega_{nb}^b$$

It is possible to rewrite the expression
 $\dot{q} = T(q)w$ such that $\dot{q} = \bar{T}(w)q$

$q[k+1]$ is computed using the matrix exponential, which serves as the *exponential map* for matrix Lie groups, ensuring an exact discretization of the quaternion differential equation:

$$q_{\text{dot}} = T_{\text{quat}}(w) * q \Rightarrow q[k+1] = \text{expm}(T_{\text{quat}}(w[k] * h) * q[k]$$

You can replace the build-in Matlab function expm.m with the custom-made MSS function expm_taylor.m for this computation.

```
function T = Tquat(u)
% Tq = Tquat(q) computes the quaternion transformation matrix Tq of
% dimension 4 x 3 for attitude such that q_dot = Tq * w
% Tw = Tquat(w) computes the quaternion transformation matrix Tw of
% dimension 4 x 4 for attitude such that q_dot = Tw * q
%
% The standard inout is the unit quaternion q = [ eta eps1 eps2 eps3 ]' of
% dimension 4. If the angular rates w = [ p q r ]' of dimension 3 are used
% as input, the alternative matrix Tw is returned.

if (length(u) == 4) % T = Tq and q = u
    eta = u(1); eps1 = u(2); eps2 = u(3); eps3 = u(4);

    T = 0.5 * [...
        -eps1 -eps2 -eps3
        eta -eps3 eps2
        eps3 eta -eps1
        -eps2 eps1 eta ];

elseif (length(u) == 3) % T = Tw and w = u
    w = [u(1) u(2) u(3)]';

    T = 0.5 * [...
        0 -w'
        w -Smtx(w) ];

end
```


2.2 从欧拉角得到单位四元数

If the Euler angles $\Theta_{nb} = [\phi, \theta, \psi]^\top$ are known the corresponding unit quaternion can be computed using the transformation (NASA 2013):

$$q_b^n = \begin{bmatrix} \cos\left(\frac{1}{2}\psi\right) \cos\left(\frac{1}{2}\theta\right) \cos\left(\frac{1}{2}\phi\right) + \sin\left(\frac{1}{2}\psi\right) \sin\left(\frac{1}{2}\theta\right) \sin\left(\frac{1}{2}\phi\right) \\ \cos\left(\frac{1}{2}\psi\right) \cos\left(\frac{1}{2}\theta\right) \sin\left(\frac{1}{2}\phi\right) - \sin\left(\frac{1}{2}\psi\right) \sin\left(\frac{1}{2}\theta\right) \cos\left(\frac{1}{2}\phi\right) \\ \sin\left(\frac{1}{2}\psi\right) \cos\left(\frac{1}{2}\theta\right) \sin\left(\frac{1}{2}\phi\right) + \cos\left(\frac{1}{2}\psi\right) \sin\left(\frac{1}{2}\theta\right) \cos\left(\frac{1}{2}\phi\right) \\ \sin\left(\frac{1}{2}\psi\right) \cos\left(\frac{1}{2}\theta\right) \cos\left(\frac{1}{2}\phi\right) - \cos\left(\frac{1}{2}\psi\right) \sin\left(\frac{1}{2}\theta\right) \sin\left(\frac{1}{2}\phi\right) \end{bmatrix}$$

Formula (2.82) is implemented in the MSS toolbox as `euler2q.m` and it is only valid for the *zyx* convention of the Euler angles. This formula can also be used to compute the initial values of the Euler parameters corresponding to Step 1 of Algorithm 2.1.

```
phi=10*(pi/180); psi = 30*(pi/180); theta=-20*(pi/180);
q = euler2q(phi,theta,psi)
q =
    0.9437
    0.1277
   -0.1449
    0.2685
>> norm(q)           % normalization test
ans =
    1
```

NASA (2013)。任务规划与分析部。欧拉角、四元数与变换矩阵。
<https://ntrs.nasa.gov/api/citations/19770024290/downloads/19770024290.pdf>

讲义：海洋船舶水动力学与运动控制 (T. I. Fossen)

2.2 从单位四元数求欧拉角

由于这两种运动学表示的旋转矩阵相等：

$$\mathbf{R}(\Theta_{nb}) := \mathbf{R}(\mathbf{q}_b^n) \quad \text{nb} \quad \phi \quad \theta \quad \psi \quad \eta \quad \varepsilon_1 \quad \varepsilon_2 \quad \varepsilon_3 \quad \top$$

$$\begin{bmatrix} R_{11} & R_{12} & R_{13} \\ R_{21} & R_{22} & R_{23} \\ R_{31} & R_{32} & R_{33} \end{bmatrix} = \begin{bmatrix} \cos\phi \cos\theta \cos\psi - \sin\phi \sin\theta \sin\psi & \cos\phi \cos\theta \sin\psi + \sin\phi \sin\theta \sin\psi & \cos\phi \sin\theta \cos\psi + \sin\phi \cos\theta \sin\psi \\ \sin\phi \cos\theta \cos\psi + \cos\phi \sin\theta \sin\psi & \sin\phi \cos\theta \sin\psi - \cos\phi \sin\theta \sin\psi & \sin\phi \sin\theta \cos\psi - \cos\phi \cos\theta \sin\psi \\ \sin\theta \cos\psi & \sin\theta \sin\psi & \cos\theta \end{bmatrix}$$

欧拉角解法：

$$\begin{aligned} \phi &= \text{atan2}(R_{32}, R_{33}) \\ \theta &= -\text{asin}(R_{31}), \quad \theta \neq \pm 90^\circ \\ \psi &= \text{atan2}(R_{21}, R_{11}) \end{aligned}$$

$$\begin{aligned} \phi &= \text{atan2}(2(\varepsilon_2 \varepsilon_3 + \varepsilon_1 \eta), 1 - 2(\varepsilon_1^2 + \varepsilon_2^2)) \\ \theta &= -\text{asin}(2(\varepsilon_1 \varepsilon_3 - \varepsilon_2 \eta)) \\ \psi &= \text{atan2}(2(\varepsilon_1 \varepsilon_2 + \varepsilon_3 \eta), 1 - 2(\varepsilon_2^2 + \varepsilon_3^2)) \end{aligned}$$

其中 $\text{atan2}(y,x)$ 是将结果限制在四象限内的反正切函数 $(-\pi, \pi]$

案例研究：运动学微分方程的数值积分

```
% Numerical integration of the kinematic differential equations.
% The goal is to compare two different methods for representing orientation:
% 1) Euler angles
% 2) Unit quaternions
% We will integrate the unit quaternions over time and transform the resulting
% quaternion into Euler angles. This result will be compared with the solution
% obtained by directly integrating the Euler angle differential equations.
```

```
clearvars;
```

```
% Simulation parameters
```

```
T_final = 50; % Final simulation time (s)
```

```
h = 0.01; % Sampling time (s)
```

```
% Initial conditions
```

```
nu = [5 -3 3 0.1 0.02 -0.02]'; % Initial velocity vector (translational and rotational)
```

```
nu_ref = zeros(6,1); % Desired velocity vector (target: no motion)
```

```
% Initial positions and orientations
```

```
pos1 = [0 0 0]'; % Initial position for Euler angle method
```

```
Theta1 = deg2rad([10 5 1])'; % Initial Euler angles [Roll, Pitch, Yaw]
```

```
pos2 = pos1; % Initial position for quaternion method
```

```
quat2 = euler2q(Theta1(1), Theta1(2), Theta1(3)); % Convert initial Euler angles to quaternion
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%% MAIN LOOP
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
for i = 1:nTimeSteps
```

```
    % Convert current quaternion to Euler angles for comparison
```

```
    [phi2, theta2, psi2] = q2euler(quat2);
```

```
    Theta2 = [phi2 theta2 psi2]';
```

```
    % Store current state data for analysis
```

```
    sim.data(i,:) = [pos1' Theta1' pos2' Theta2' nu'];
```

```
% Numerical integration of Euler angle differential equations (Method 1)
```

```
% Position update using forward Euler method
```

```
pos1 = pos1 + h * Rzyx(Theta1(1), Theta1(2), Theta1(3)) * nu(1:3);
```

```
% Euler angles update using forward Euler method
```

```
Theta1 = Theta1 + h * Tzyx(Theta1(1), Theta1(2)) * nu(4:6);
```

```
% Numerical integration of quaternion differential equations (Method 2)
```

```
% Position update using forward Euler method
```

```
pos2 = pos2 + h * Rquat(quat2) * nu(1:3);
```

```
% Quaternion update using exact discretization
```

```
quat2 = expm(Tquat(nu(4:6)) * h) * quat2;
```

```
quat2 = quat2 / norm(quat2); % Ensure quaternion remains normalized
```

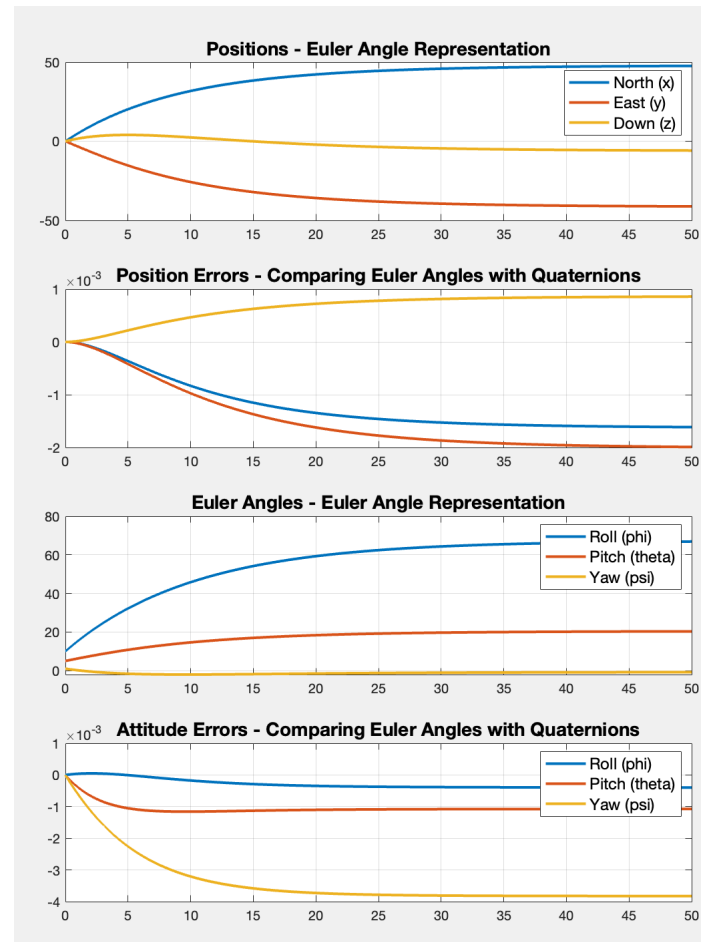
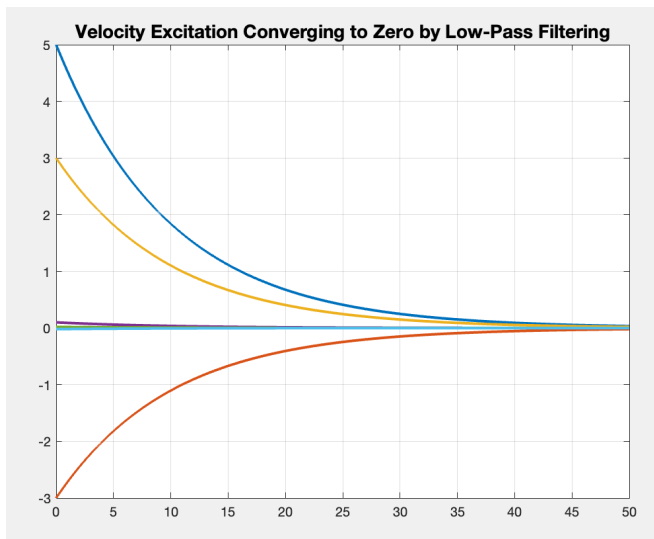
```
% Update velocities using low-pass filter
```

```
w_n = 0.1; % LP-filter natural frequency
```

```
nu = lowPassFilter(nu, nu_ref, w_n, h);
```

```
end
```

案例研究：运动学微分方程的数值积分（续）



2.3 ECEF 与 NED 之间的变换

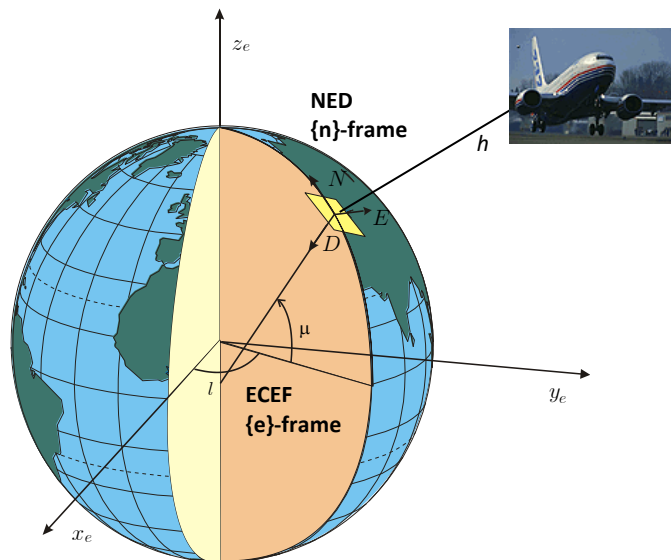
地球表面上或以上的一个点可以被唯一确定如下：

经度： l (度) 纬度： μ (度) 椭球高： h (米)

NED 轴定义：

N – 北轴指向北

E – 东轴指向东 D – 下轴指向地面法线方向向下



2.3 经度和纬度转换

ECEF 和 NED 速度向量之间的变换是：

$$\dot{\mathbf{p}}_{eb}^e = \mathbf{R}_n^e \dot{\mathbf{p}}_{eb}^n = \mathbf{R}_n^e \mathbf{R}_b^n \mathbf{v}_{eb}^b$$

两个主要旋转：

1. 绕 z 轴旋转 l
2. 绕 y 轴旋转 $-\mu - \pi/2$ 。

$$\begin{aligned} \mathbf{R}_n^e &:= \mathbf{R}_{z,l} \mathbf{R}_{y,-\mu-\frac{\pi}{2}} \\ &= \begin{bmatrix} \cos(l) & -\sin(l) & 0 \\ \sin(l) & \cos(l) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos(-\mu-\frac{\pi}{2}) & 0 & \sin(-\mu-\frac{\pi}{2}) \\ 0 & 1 & 0 \\ -\sin(-\mu-\frac{\pi}{2}) & 0 & \cos(-\mu-\frac{\pi}{2}) \end{bmatrix} \end{aligned}$$

$$\mathbf{R}_n^e = \mathbf{R}(\Theta_{en}) = \begin{bmatrix} -\cos(l) \sin(\mu) & -\sin(l) & -\cos(l) \cos(\mu) \\ -\sin(l) \sin(\mu) & \cos(l) & -\sin(l) \cos(\mu) \\ \cos(\mu) & 0 & -\sin(\mu) \end{bmatrix}$$

Matlab:

The rotation matrix $\mathbf{R}_n^e$ is computed using the MSS toolbox command

```
R = Rll(1,mu)
```

2.3 从 ECEF 坐标获取经纬度

卫星导航系统的测量值是在ECEF坐标系中给出的：对操作员没有用。

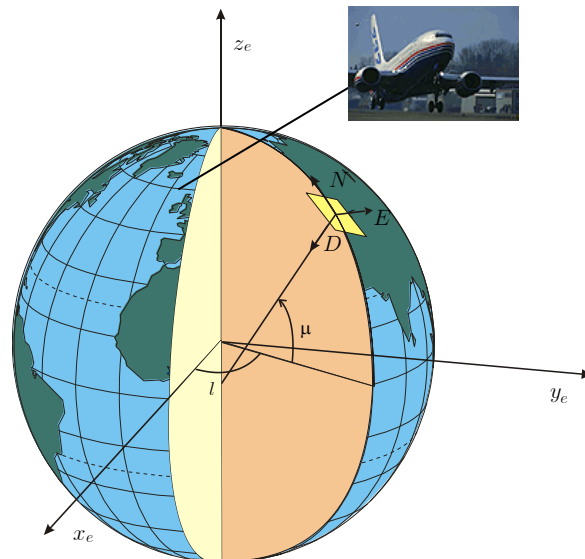
因此，地面位置数据的呈现是以椭圆参数的经度 λ 、纬度 μ 和高度 h 来表示的。

$$\mathbf{p}_{eb}^e = [x^e, y^e, z^e]^\top$$



Transformation

λ, μ and h



2.3 从ECEF坐标求经度/纬度（续）

Parameters	Comments
$r_e = 6\,378\,137\text{ m}$	Equatorial radius of ellipsoid (semimajor axis)
$r_p = 6\,356\,752\text{ m}$	Polar axis radius of ellipsoid (semiminor axis)
$\omega_e = 7.292115 \cdot 10^{-5}\text{ rad/s}$	Angular velocity of the Earth
$e = 0.0818$	Eccentricity of ellipsoid

$$N = \frac{r_e^2}{\sqrt{r_e^2 \cos^2 \mu + r_p^2 \sin^2 \mu}} \quad e = \sqrt{1 - \left(\frac{r_p}{r_e}\right)^2}$$

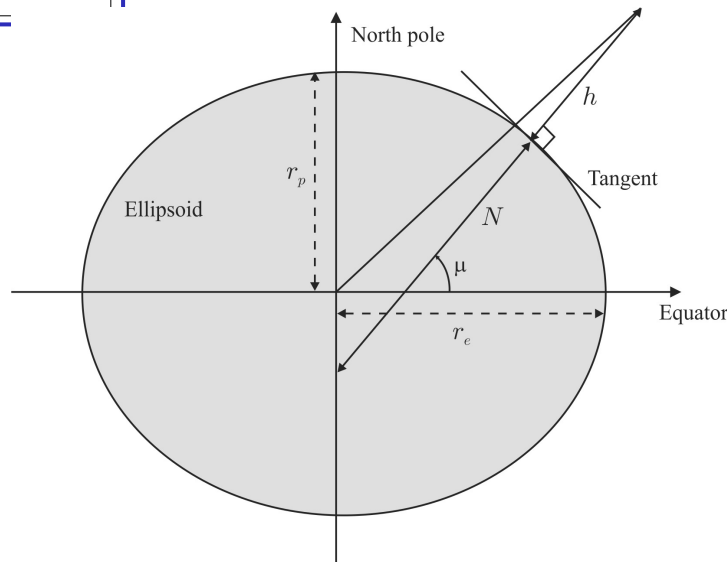
$$l = \text{atan}\left(\frac{y^e}{x^e}\right) \quad \mathbf{p}_{eb}^e = [x^e, y^e, z^e]^\top$$

当纬度 μ 和高度 h 被隐式计算时

$$\tan(\mu) = \frac{z^e}{p} \left(1 - e^2 \frac{N}{N + h}\right)^{-1}$$

$$h = \frac{p}{\cos(\mu)} - N$$

WGS-84



世界大地测量系统（1984）。其定义及与地方大地测量系统的关系。DMA TR 8350.2, 第2版, 美国弗吉尼亚州费尔法克斯, 国防测绘局。

讲义：海洋船舶水动力学与运动控制（T. I. Fossen）

2.3 从ECEF坐标求经度/纬度（续）

Algorithm 2.2 (Transformation of (x^e, y^e, z^e) to (l, μ, h))

1. Compute $p = \sqrt{(x^e)^2 + (y^e)^2}$.
2. Compute the approximate value μ_0 from

$$\tan(\mu_0) = \frac{z^e}{p} (1 - e^2)^{-1}$$

3. Compute an approximate value N from

$$N = \frac{r_e^2}{\sqrt{r_e^2 \cos^2(\mu_0) + r_p^2 \sin^2(\mu_0)}}$$

4. Compute the ellipsoidal height by

$$h = \frac{p}{\cos(\mu_0)} - N$$

5. Compute an improved value for the latitude by

$$\tan(\mu) = \frac{z^e}{p} \left(1 - e^2 \frac{N}{N + h} \right)^{-1}$$

6. Check for another iteration step: if $|\mu - \mu_0| < \varepsilon$, where ε is a small number, then the iteration is complete. Otherwise set $\mu_0 = \mu$ and continue with Step 3.

Matlab:

Algorithm 2.2 is programmed in the MSS toolbox as a function

```
[l, mu, h] = ecef2llh(x, y, z)
```

霍夫曼-韦伦霍夫, B., H. 利希滕格和 J. 柯林斯 (1994)。全球定位系统: 理论与实践。第3版。施普林格出版社。纽约, 纽约州。

2.3 从经纬度到地心地固坐标

从给定高度 h 的转换到 e 系 μ 系

$$\mathbf{p}_{eb}^e = [x^e, y^e, z^e]^\top$$

由 (Heiskanen 和 Moritz 1967) 给出

$$\begin{bmatrix} x^e \\ y^e \\ z^e \end{bmatrix} = \begin{bmatrix} (N + h) \cos(\mu) \cos(l) \\ (N + h) \cos(\mu) \sin(l) \\ \left(\frac{r_p^2}{r_e^2} N + h \right) \sin(\mu) \end{bmatrix}$$

Heiskanen, W. A. and H. Moritz (1967). Physical Geodesy. Freeman. London.

2.3 从经纬度到地心地固坐标（续）

Matlab:

The transformation from $\Theta_{en} = [l, \mu]^\top$ to $p_{eb}^e = [x^e, y^e, z^e]^\top$. Equation (2.96), is programmed in the MSS toolbox function

```
[x,y,z] = llh2ecef(l,mu,h)
```

Assume that $l = 10.3^\circ$, $\mu = 63.0^\circ$ and $h = 0$ m. Hence, the ECEF coordinates are computed to be

$$\begin{bmatrix} x^e \\ y^e \\ z^e \end{bmatrix} = \begin{bmatrix} 2\,856\,552 \text{ m} \\ 519\,123 \text{ m} \\ 5\,659\,978 \text{ m} \end{bmatrix}$$

using the MSS Matlab command

```
[x,y,z] = llh2ecef(10.3*(pi/180),63.0*(pi/180),0)
```

2.4 ECEF 与平地坐标之间的变换

对于本地平面地球导航，可以假设NED切平面固定在地球表面

假设 NED 切平面位于 l_0 和 μ_0 ，使得

$$\mathbf{R}_n^e = \mathbf{R}(\boldsymbol{\theta}_{en}) = \text{constant} \quad \boldsymbol{\theta}_{en} = [l_0, \mu_0]^\top$$

ECEF 坐标满足微分方程

$$\dot{\mathbf{p}}_{eb}^e = \mathbf{R}_n^e \mathbf{R}_b^n \mathbf{v}_{eb}^b$$

平面地球是对在有限区域内运行的船舶和漂浮结构的一个良好近似。

平面地球是海洋船舶全球航点跟踪控制系统的一个不良近似，因为 (l, μ) 在船舶穿越不同大陆之间时会有很大变化。

2.4 平面地球上的经度、纬度和高度坐标

给定一个局部NED位置，其坐标原点为 (l_0, μ_0) ，参考高度为 h_{ref} 米（地球表面以上），经度和纬度的变化为（方程2.39, Farrell 2008）

$$\Delta l = \frac{y^n}{(R_N + h_{\text{ref}}) \cos(\mu_0)}$$

$$\Delta \mu = \frac{x^n}{R_M + h_{\text{ref}}}$$

$$\Delta l := l - l_0$$

$$\Delta \mu := \mu - \mu_0$$

```
% WGS-84 parameters
a = 6378137; % Semi-major axis (equatorial radius)
f = 1 / 298.257223563; % Flattening
e = sqrt( 2*f - f^2 ); % Earth eccentricity
```

$$R_N = \frac{r_e}{\sqrt{1 - e^2 \sin^2(\mu_0)}}$$

$$R_M = R_N \frac{1 - e^2}{1 - e^2 \sin^2(\mu_0)}$$

$$l = \text{ssa}(l_0 + \Delta l)$$

$$\mu = \text{ssa}(\mu_0 + \Delta \mu)$$

$$h = h_{\text{ref}} - z^n$$

Matlab:

The `ssa(·)` function is implemented in the MSS toolbox as

```
angle = ssa(angle,type)
```

where `type` can be chosen as `'rad'` or `'deg'`.

`ssa` 是将幅角限制在区间 $[-\pi, \pi)$ 内的最小带符号角

Matlab:

The triplet (l, μ, h) corresponding to the NED coordinates (x^n, y^n, z^n) in a flat-Earth coordinate origin located at (l_0, μ_0) with reference height h_{ref} in meters above the ellipsoid is computed as

```
[l,mu,h] = flat2llh(x,y,z,l0,mu0,h_ref)
```

最小有符号角：两个角之间的最小差

```
function angle = ssa(angle,unit)
% SSA is the "smallest signed angle" or the smallest difference between two
% angles. Examples:
%
% angle = ssa(angle) maps an angle in rad to the interval [-pi pi]
% angle = ssa(angle,'deg') maps an angle in deg to the interval [-180 180]
%
% For feedback control systems and state estimators used to control the
% attitude of vehicles, the difference of two angles should always be
% mapped to [-pi pi] or [-180 180] to avoid step inputs/discontinuities.
%
% Note that in many languages (C, C++, C#, JavaScript), the modulus
% operator mod(x,y) returns a value with the same sign as x.
% For these use a custom mod function: mod(x,y) = x - floor(x/y) * y
% For the Unity game engine use: Mathf.DeltaAngle
%
% Author:      Thor I. Fossen
% Date:        2018-09-21
% Revisions:   2020-03-04 Deafult rad, otional argument for deg
%


---


if (nargin == 1)
    angle = mod( angle + pi, 2 * pi ) - pi;
elseif strcmp(unit,'deg')
    angle = mod( angle + 180, 360 ) - 180;
end
```

Command Window

```
>> ssa(181,'deg')
ans =
    -179
>> ssa(179,'deg')
ans =
    179
fx >> |
```

2.4 从经度、纬度和高度计算平面地球坐标

相对于以原点 (l_0, μ_0) 和参考高度 h_{ref} 的平地坐标系，NED 位置 (x_n, y_n, z_n) 的计算公式为（方程 2.39, Farrell 2008）

$$\begin{aligned}\Delta l &:= l - l_0 \\ \Delta \mu &:= \mu - \mu_0\end{aligned}$$



$$\begin{aligned}x^n &= \Delta \mu (R_M + h_{\text{ref}}) \\ y^n &= \Delta l (R_N + h_{\text{ref}}) \cos(\mu_0) \\ z^n &= h_{\text{ref}} - h\end{aligned}$$

$$\begin{aligned}R_N &= \frac{r_e}{\sqrt{1 - e^2 \sin^2(\mu_0)}} \\ R_M &= R_N \frac{1 - e^2}{1 - e^2 \sin^2(\mu_0)}\end{aligned}$$

Matlab:

For a marine craft operating on the sea surface ($h = 0$) or submerged ($h < 0$) with longitude and latitude (l, μ) and a local coordinate origin at $(l_0, \mu_0, h_{\text{ref}})$, the NED positions can be computed using the MSS toolbox command

```
[x, y, z] = llh2flat(l, mu, h, l0, mu0, h_ref)
```

2.5 回顾：东北-下 (NED) 微分方程

欧拉角旋转矩阵表示

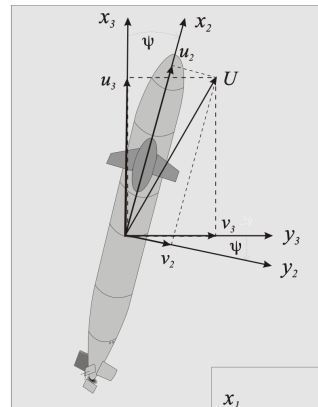
$$\dot{\mathbf{p}}^n = \mathbf{R}_b^n \mathbf{v}^b$$

$$\mathbf{p}^n = [x^n, y^n, z^n]^\top$$

$$\mathbf{v}^b = [u, v, w]^\top$$

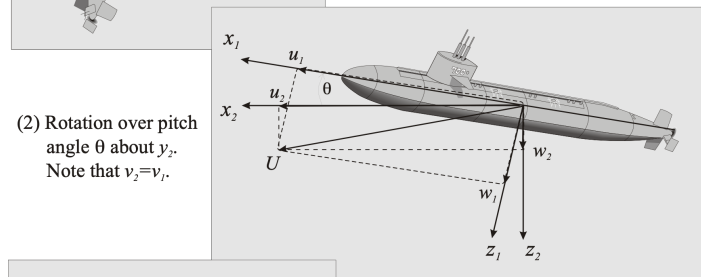
$$\mathbf{R}_b^n = \begin{bmatrix} c\psi c\theta & -s\psi c\theta + c\psi s\theta s\phi & s\psi c\theta + c\psi c\phi s\theta \\ s\psi c\theta & c\psi c\theta + s\phi s\theta s\psi & -c\psi s\theta + s\theta s\psi c\phi \\ -s\theta & c\theta s\phi & c\theta c\phi \end{bmatrix}$$

$$\begin{aligned} \dot{x}^n &= u \cos(\psi) \cos(\theta) \\ &\quad + v (\cos(\psi) \sin(\theta) \sin(\phi) - \sin(\psi) \cos(\phi)) \\ &\quad + w (\sin(\psi) \sin(\phi) + \cos(\psi) \cos(\phi) \sin(\theta)) \\ \dot{y}^n &= u \sin(\psi) \cos(\theta) \\ &\quad + v (\cos(\psi) \cos(\phi) + \sin(\phi) \sin(\theta) \sin(\psi)) \\ &\quad + w (\sin(\theta) \sin(\psi) \cos(\phi) - \cos(\psi) \sin(\phi)) \\ \dot{z}^n &= -u \sin(\theta) \\ &\quad + v \cos(\theta) \sin(\phi) \\ &\quad + w \cos(\theta) \cos(\phi) \end{aligned}$$

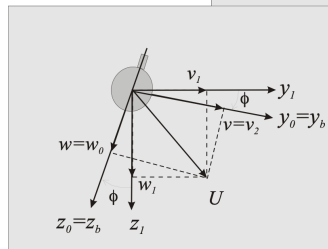


(1) Rotation over yaw angle ψ about z_3 . Note that $w_3 = w_2$.

The ZZX sequence is the proper Euler angle rotation sequence used in robotics, but for aircraft and spacecraft, we use the Tait-Bryan ZYX sequence (yaw-pitch-roll), so that the singularity occurs at ± 90 degrees in pitch.



(2) Rotation over pitch angle θ about y_2 . Note that $v_2 = v_1$.



(3) Rotation over roll angle ϕ about x_1 . Note that $u_1 = u_2$.

2.5 Two-Dimensional Amplitude-Phase Form

We want to prove the famous relationship (from the figure)

Course angle = Heading (yaw) angle + Horizontal Crab angle

$$\chi = \psi + \beta_c$$

$$\beta_c = \sin^{-1} \left(\frac{v}{U} \right)$$

Horizontal crab angle

Proof:

$$\dot{x}^n = u \cos(\psi) - v \sin(\psi)$$

$$\dot{y}^n = u \sin(\psi) + v \cos(\psi)$$

These equations can be expressed in amplitude-phase form

$$\dot{x}^n = U \cos(\psi + \beta_c) := U \cos(\chi)$$

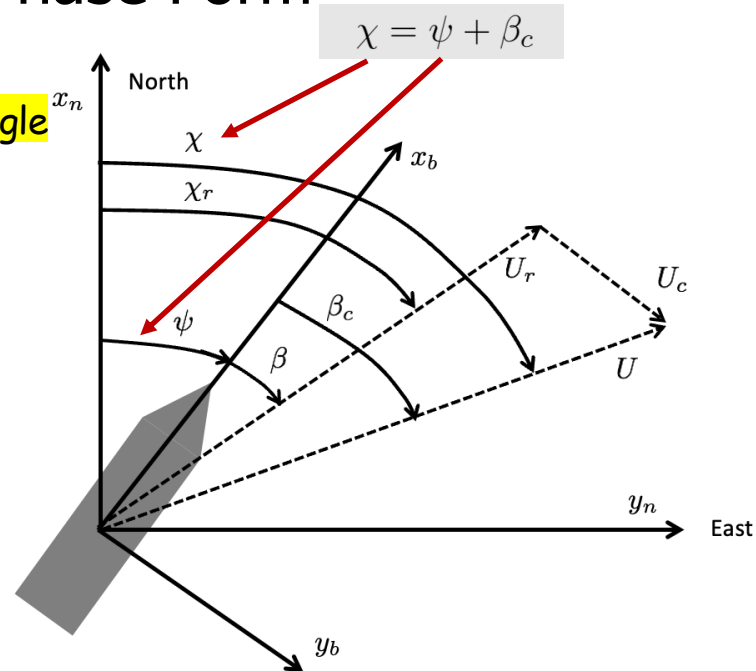
$$\dot{y}^n = U \sin(\psi + \beta_c) := U \sin(\chi)$$

Amplitude (speed):

$$U = \sqrt{u^2 + v^2}$$

Phase (horizontal crab angle):

$$\beta_c = \sin^{-1} \left(\frac{v}{U} \right)$$



$$\begin{aligned} y_1 &= c_1 \sin(x) + c_2 \cos(x) \\ &= A \sin(x) \cos(\varphi) + A \cos(x) \sin(\varphi) \\ &= A \sin(x + \varphi) \end{aligned}$$

$$c_1 = A \cos(\varphi), \quad c_2 = A \sin(\varphi)$$

$$\begin{aligned} A &= \sqrt{c_1^2 + c_2^2} \\ \varphi &= \sin^{-1} \left(\frac{c_2}{A} \right) \end{aligned}$$

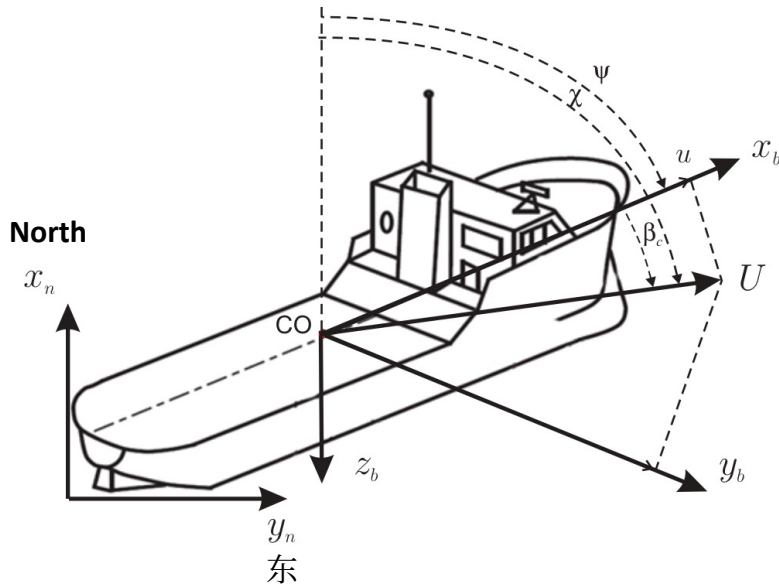
2.5 航向、航迹和偏航角的定义

航向角、航迹角和偏航角之间的关系对于在水平面上操纵车辆非常重要。

在大量关于制导、导航和控制的文献中，术语“航向”和“方位”常被互换使用，这造成了混淆。

- 航向角 χ 航向角 χ 指车辆移动的基本方向。使用全球导航卫星系统 (GNSS) (或水下的HPR) 测量
- 航向 (偏航) 角 ψ 航向角 ψ 是飞行器船首 (x_b 轴) 指向的方向。使用指南针测量
- 水平侧滑角 β_c 对图形的检查确认，侧滑角是航向角与船首方向角之间的差值：

$$\chi = \psi + \beta_c \quad \beta_c = \sin^{-1} \left(\frac{v}{U} \right)$$



2.5 由侧风分量引起的水平横摇角

一架 B777（如照片所示）在接近并降落于最大示范横风分量为 38 节的情况下，通常的最大横滑角约为 16 度。

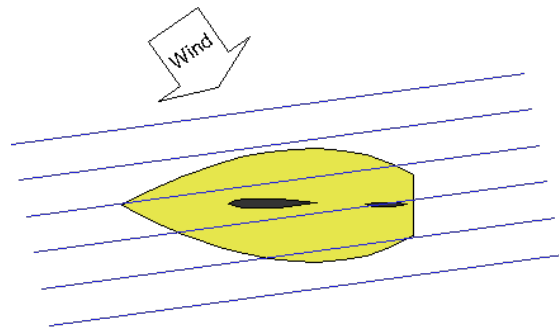
$$\chi = \psi + \beta_c$$

$$\beta_c = \sin^{-1} \left(\frac{v}{U} \right)$$

物理观测角是水平偏航角 β_c ，而不是侧滑角 β



受到风、浪和洋流影响的海上船只也会有非零的水平偏转角。



<https://aviation.stackexchange.com/questions/58861/飞机着陆时与跑道中心线之间的最大角度是多少>

2.5 海洋洋流的扩展：攻角（AOA）和侧滑角（SSA）

对于暴露在洋流中的海上船舶，引入了相对速度的概念。
相对速度是

$$u_r = u - u_c \quad U_r = \sqrt{u_r^2 + v_r^2 + w_r^2} \quad \text{相对速度}$$

$$v_r = v - v_c$$

$$w_r = w - w_c$$

其中 u_c 、 v_c 和 w_c 是当前速度，并且

$$U_c = \sqrt{u_c^2 + v_c^2 + w_c^2}$$

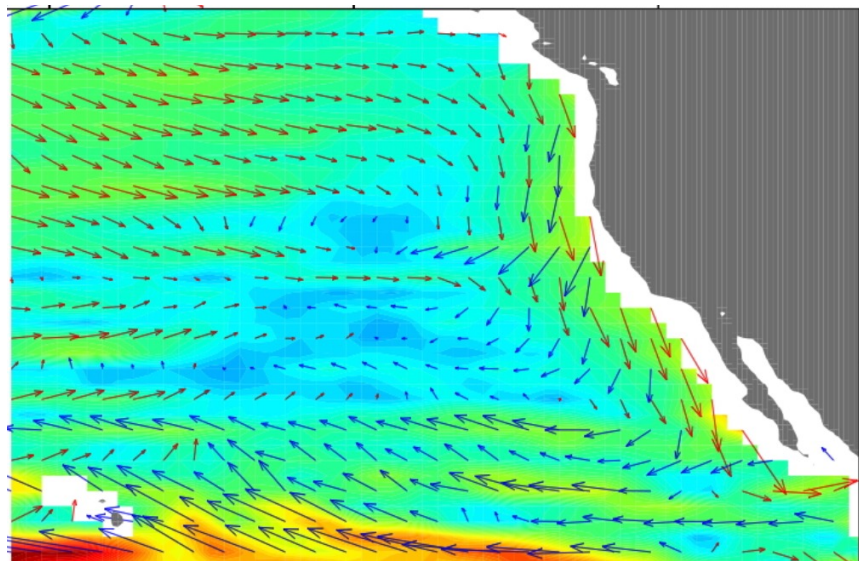
是海流速度。

AOA 和 SSA

$$\alpha = \text{atan2}(w_r, u_r)$$

$$\beta = \sin^{-1}\left(\frac{v_r}{U_r}\right)$$

AOA 和 SSA 主要用于从风洞数据中获取升力和阻力系数，但也可用于控制系统设计。



2.5 水平侧滑角与横向偏航角

飞机在风中运行，而船舶可能会受到洋流、波浪和风的影响。空气动力和水动力的作用力是相对速度的函数：

$$u_r = u - u_f$$

$$v_r = v - v_f$$

The subscript f denotes the flow due to wind, waves, and/or ocean currents

升力将垂直于相对流，而阻力将平行于相对流。二维线性相对速度可以表示为：

$$u_r = U_r \cos(\beta)$$

$$v_r = U_r \sin(\beta)$$

$$U_r = (u_r^2 + v_r^2)^{1/2}$$

侧滑角

$$\beta = \sin^{-1} \left(\frac{v_r}{U_r} \right)$$

注意，当侧滑角存在时，水平螃蟹角等

$$u_f = v_f = 0$$

$$\beta_c = \sin^{-1} \left(\frac{v}{U} \right)$$

- 水平螃蟹角是指车辆由于外部力（如洋流、海浪或风）影响而移动的方向 χ 与车辆航向 ψ 之间的角度，满足 $\beta_c = \chi - \psi$
- 侧滑角是车辆 x_b 轴与流速方向之间的角度。

在文献中，术语“侧滑角”通常用于表示“水平偏航角”，而我们明确区分这两个角度。侧滑角用于计算空气动力/水动力系数（风洞查表），而水平偏航角则用于视线指导律。

2.5 三维幅度-相位表示

$$\dot{\mathbf{p}}^n = \mathbf{R}_b^n \mathbf{v}^b$$

航向角 χ 和飞行路径角 γ 可以描述 NED 速度向量在三维空间中的情况

NED运动学微分方程 旋转

$$\chi = \text{atan2}(\dot{y}^n, \dot{x}^n), \quad \gamma = \sin^{-1}\left(\frac{-\dot{z}^n}{U}\right)$$

关于 y 和 z 轴的速度向量 $[U, 0, 0]$ 并使用水平速度

$$\begin{aligned} \dot{\mathbf{p}}^n &= \mathbf{R}_z(\chi) \mathbf{R}_y(\gamma) \begin{bmatrix} U \\ 0 \\ 0 \end{bmatrix} \\ &= U \begin{bmatrix} \cos(\gamma) \cos(\chi) \\ \cos(\gamma) \sin(\chi) \\ -\sin(\gamma) \end{bmatrix} \\ &= \begin{bmatrix} U_h \cos(\psi + \beta_c) \\ U_h \sin(\psi + \beta_c) \\ -U \sin(\theta - \alpha_c) \end{bmatrix} \end{aligned}$$



端

$$U_h := U \cos(\gamma)$$

Coates, E. M. 和 T. I. Fossen (2025). 用于三维自适应视线 (ALOS) 制导的球面振幅-相位公式及其 USGES 稳定性保证。已提交至《Automatica》。开放获取: arxiv.org/abs/2505.08344

2.5 三维幅度-相位表示

欧拉角旋转矩阵表示

$$\dot{\mathbf{p}}^n = \mathbf{R}_b^n \mathbf{v}^b$$

$$\mathbf{p}^n = [x^n, y^n, z^n]^\top$$

$$\mathbf{v}^b = [u, v, w]^\top$$

$$\mathbf{R}_b^n = \begin{bmatrix} c\psi c\theta & -s\psi c\theta + c\psi s\theta s\phi & s\psi s\theta + c\psi c\theta s\phi \\ s\psi c\theta & c\psi c\theta + s\psi s\theta s\phi & -c\psi s\theta + s\psi c\theta s\phi \\ -s\theta & c\theta s\phi & c\theta c\phi \end{bmatrix}$$

Lecture Notes: Marine Craft Hydrodynamics and Motion Control (T. I. Fossen)

$$\begin{aligned} \dot{x}^n &= u \cos(\psi) \cos(\theta) \\ &+ v (\cos(\psi) \sin(\theta) \sin(\phi) - \sin(\psi) \cos(\phi)) \\ &+ w (\sin(\psi) \sin(\phi) + \cos(\psi) \cos(\phi) \sin(\theta)) \\ \dot{y}^n &= u \sin(\psi) \cos(\theta) \\ &+ v (\cos(\psi) \cos(\phi) + \sin(\phi) \sin(\theta) \sin(\psi)) \\ &+ w (\sin(\theta) \sin(\psi) \cos(\phi) - \cos(\psi) \sin(\phi)) \\ \dot{z}^n &= -u \sin(\theta) \\ &+ v \cos(\theta) \sin(\phi) \\ &+ w \cos(\theta) \cos(\phi) \end{aligned}$$

47

Spherical Amplitude-Phase Representation

$$\begin{bmatrix} \dot{x}^n \\ \dot{y}^n \\ \dot{z}^n \end{bmatrix} = \begin{bmatrix} U_h \cos(\psi + \beta_c) \\ U_h \sin(\psi + \beta_c) \\ -U \sin(\theta - \alpha_c) \end{bmatrix}$$

Control inputs:

- Pitch angle, θ
- Yaw angle, ψ
- Speed, U

Course angle

$$\chi = \psi + \beta_c$$

Flight-path angle

$$\gamma := \theta - \alpha_c$$

Horizontal speed

$$U_h := U \cos(\gamma)$$

Vertical and horizontal crab angles

$$\alpha_c = \theta - \sin^{-1} \left(\frac{u \sin(\theta) - [v \sin(\phi) + w \cos(\phi)] \cos(\theta)}{U} \right)$$

$$\beta_c = \tan^{-1} \left(\frac{v \cos(\phi) - w \sin(\phi)}{u \cos(\theta) + [v \sin(\phi) + w \cos(\phi)] \sin(\theta)} \right)$$

Coates, E. M. and T. I. Fossen (2025). A Spherical Amplitude-Phase Formulation for 3-D Adaptive Line-of-Sight (ALOS) Guidance with USGES Stability Guarantees. Submitted to Automatica. Open Access: arxiv.org/abs/2505.08344

2.5 BODY 与 FLOW 之间的转换

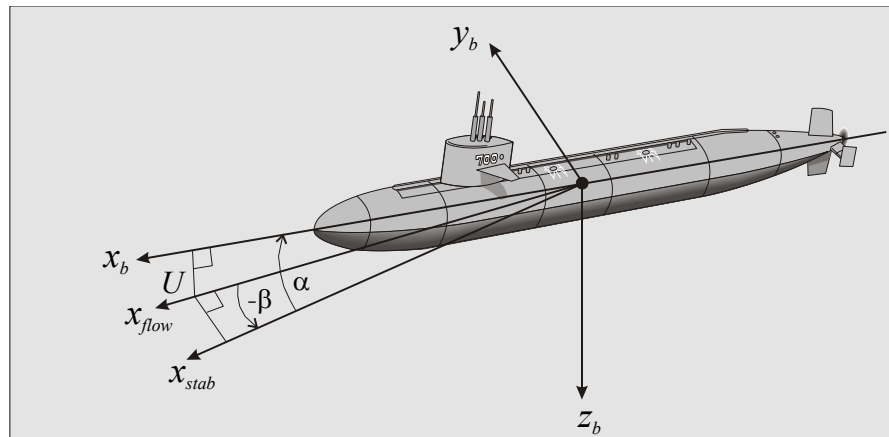
FLOW 轴通常用于表示流体动力学数据。通过旋转 BODY 轴系统找到 FLOW 轴，使得得到的 x 轴与自由流方向平行。

在 FLOW 坐标轴中， x 轴直接指向相对流动方向，而 z 轴保持在参考平面内，但会旋转以保持与 x 轴垂直。 y 轴则完成右手坐标系。

$$\begin{bmatrix} X \\ Z \end{bmatrix} = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix} \begin{bmatrix} -F_{\text{drag}} \\ -F_{\text{lift}} \end{bmatrix}$$

$$F_{\text{drag}} = \frac{1}{2} \rho V_r^2 S C_D(\alpha)$$

$$F_{\text{lift}} = \frac{1}{2} \rho V_r^2 S C_L(\alpha)$$



2.5 BODY与FLOW之间的旋转矩阵

$$\mathbf{v}_r^{\text{stab}} = \mathbf{R}_{y,\alpha} \mathbf{v}_r^b$$

$$\mathbf{v}_r^{\text{flow}} = \mathbf{R}_{z,-\beta} \mathbf{v}_r^{\text{stab}}$$

速度变换:

$$\begin{bmatrix} u_r \\ v_r \\ w_r \end{bmatrix} = \mathbf{R}_{y,\alpha}^\top \mathbf{R}_{z,-\beta}^\top \begin{bmatrix} U_r \\ 0 \\ 0 \end{bmatrix}$$

主旋转:

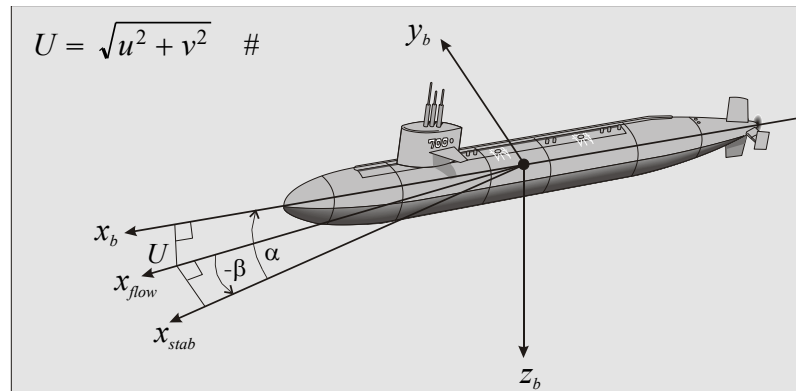
$$\mathbf{R}_{y,\alpha} = \begin{bmatrix} \cos(\alpha) & 0 & \sin(\alpha) \\ 0 & 1 & 0 \\ -\sin(\alpha) & 0 & \cos(\alpha) \end{bmatrix}$$

$$\mathbf{R}_{z,-\beta} = \mathbf{R}_{z,\beta}^\top = \begin{bmatrix} \cos(\beta) & \sin(\beta) & 0 \\ -\sin(\beta) & \cos(\beta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{v}_r^{\text{flow}} = \mathbf{R}_b^{\text{flow}} \mathbf{v}_r^b$$

$$\mathbf{v}_r^b = (\mathbf{R}_b^{\text{flow}})^\top \mathbf{v}_r^{\text{flow}}$$

$$\mathbf{R}_b^{\text{flow}} = \mathbf{R}_{z,-\beta} \mathbf{R}_{y,\alpha} = \begin{bmatrix} \cos(\beta) \cos(\alpha) & \sin(\beta) & \cos(\beta) \sin(\alpha) \\ -\sin(\beta) \cos(\alpha) & \cos(\beta) & -\sin(\beta) \sin(\alpha) \\ -\sin(\alpha) & 0 & \cos(\alpha) \end{bmatrix}$$



2.5 相对速度作为攻角、SSA 和速度的函数

$$\mathbf{v}_r^{\text{flow}} = \mathbf{R}_b^{\text{flow}} \mathbf{v}_r^b \quad \longrightarrow \quad \mathbf{v}_r^b = (\mathbf{R}_b^{\text{flow}})^\top \mathbf{v}_r^{\text{flow}} \quad \longrightarrow \quad \begin{bmatrix} u_r \\ v_r \\ w_r \end{bmatrix} = \mathbf{R}_{y,\alpha}^\top \mathbf{R}_{z,-\beta}^\top \begin{bmatrix} U_r \\ 0 \\ 0 \end{bmatrix}$$

$$\mathbf{R}_b^{\text{flow}} = \mathbf{R}_{z,-\beta} \mathbf{R}_{y,\alpha} = \begin{bmatrix} \cos(\beta) \cos(\alpha) & \sin(\beta) & \cos(\beta) \sin(\alpha) \\ -\sin(\beta) \cos(\alpha) & \cos(\beta) & -\sin(\beta) \sin(\alpha) \\ -\sin(\alpha) & 0 & \cos(\alpha) \end{bmatrix}$$

分量形式的相对速度

$$u_r = U_r \cos(\alpha) \cos(\beta)$$

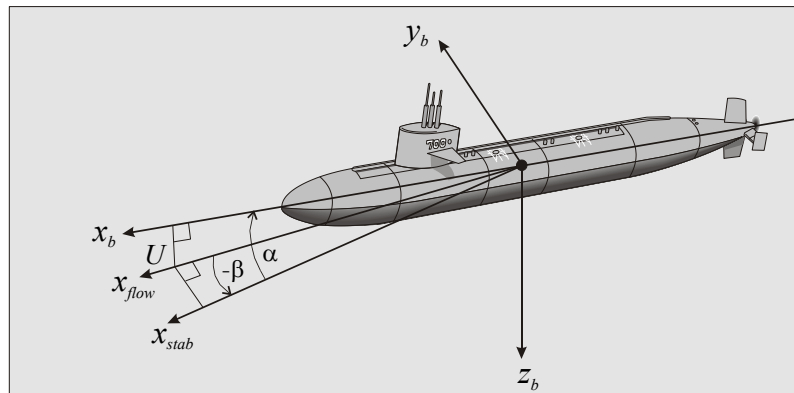
$$v_r = U_r \sin(\beta)$$

$$w_r = U_r \sin(\alpha) \cos(\beta)$$

AOA 和 SSA

$$\alpha = \tan^{-1} \left(\frac{w_r}{u_r} \right)$$

$$\beta = \sin^{-1} \left(\frac{v_r}{U_r} \right)$$



章节目标 - 再访

- 理解地理坐标系NED、地心坐标系ECEF以及机体固定坐标系BODY。
- 了解什么是FLOW轴以及为什么我们在海上船舶和飞机上使用这些轴
- 能够写出将机体速度与NED位置相关联的微分方程，包括欧拉角和单位四元数的情况。
- 能够：
 - 将 ECEF (x^e, y^e, z^e) 位置转换为（经度，纬度，高度），反之亦然
 - 将（经度，纬度，高度）转换为平面地球坐标 (x^n, y^n, z^n)，反之亦然
- 定义、可视化并解释以下的用途：
 - 攻角 α
 - 侧滑角 β
 - 垂直螃蟹角 α_c
 - 水平螃蟹角 β_c
 - 航向角 ψ
 - 航向角 χ